HDOC Day-71

Aim

Write a C program to dynamically allocate two matrices, perform addition, subtraction and multiplication using pointer notation, implement each operation in a separate function, and free all allocated memory.

C Program

```
#include <stdio.h>
#include <stdlib.h>

void add(int *a, int *b, int *c, int n) {
    for (int i = 0; i < n*n; i++)
        *(c+i) = *(a+i) + *(b+i);
}

void subtract(int *a, int *b, int *c, int n) {
    for (int i = 0; i < n*n; i++)
        *(c+i) = *(a+i) - *(b+i);
}

void multiply(int *a, int *b, int *c, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            *(c+i*n+j) = 0;
            for (int k = 0; k < n; k++)
                *(c+i*n+j) += *(a+i*n+k) * *(b+k*n+j);
        }
    }
}

void display(int *a, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", *(a+i*n+j));
        printf("\n");
    }
}

int main(void) {
    int n;
    printf("Enter order of square matrices: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid order.\n");
        return 1;
    }
    if ((size_t)n > 1000) {
        printf("Order too large.\n");
        return 1;
    }

    size_t count = (size_t)n * n;
    int *a = malloc(count * sizeof(int));
    int *b = malloc(count * sizeof(int));
    int *c = malloc(count * sizeof(int));
    if (a == NULL || b == NULL || c == NULL) {
        printf("Memory allocation failed.\n");
        free(a); free(b); free(c);
        return 1;
    }

    printf("Enter elements of matrix A:\n");
    for (size_t i = 0; i < count; i++) {
        if (scanf("%d", a+i) != 1) goto input_error;
```

```
    }
    printf("Enter elements of matrix B:\n");
    for (size_t i = 0; i < count; i++) {
        if (scanf("%d", b+i) != 1) goto input_error;
    }

    add(a, b, c, n);
    printf("Addition:\n"); display(c, n);
    subtract(a, b, c, n);
    printf("Subtraction (A-B):\n"); display(c, n);
    multiply(a, b, c, n);
    printf("Multiplication:\n"); display(c, n);

    free(a); free(b); free(c);
    return 0;

input_error:
    printf("Invalid matrix element.\n");
    free(a); free(b); free(c);
    return 1;
}
```

Sample Input and Output

```
Enter order of square matrices: 2
Enter elements of matrix A:
1 2 3 4
Enter elements of matrix B:
5 6 7 8
Addition:
6 8
10 12
Subtraction (A-B):
-4 -4
-4 -4
Multiplication:
19 22
43 50
```

Result

The program successfully performs matrix addition, subtraction and multiplication using dynamically allocated memory and pointer notation. All allocated memory is released using free().